

Valores faltantes y duplicados: decidir qué limpiar y qué conservar

Valores faltantes y duplicados: decidir qué limpiar y qué conservar

Bienvenido a la primera unidad del módulo de **Manipulación de Datos con Pandas**. En este punto de tu camino como científico de datos, ya sabes cargar un archivo y echar un vistazo rápido a su contenido. Sin embargo, en el mundo real, los datos rara vez vienen "listos para usar". Imagina que recibes un listado de ventas mensual y descubres que algunos clientes no tienen registrada su edad, otros aparecen tres veces con la misma compra, y en algunas filas simplemente no hay información.

¿Deberías borrar esos registros? ¿O tal vez inventar un valor lógico para completar los huecos? Esta es la etapa de **Limpieza de Datos**, y es donde se toma una de las decisiones más críticas: ¿con qué nos quedamos y qué descartamos? Limpiar mal un dataset es como construir un edificio sobre cimientos de arena; no importa qué tan avanzado sea tu modelo de Inteligencia Artificial después, los resultados serán inestables si la base está sucia.

1. El lenguaje de la ausencia: NaN, Null, None y Vacío

En programación y bases de datos, existen diferentes formas de decir que algo "no está". Aunque para nosotros todos signifiquen "ausencia", Pandas los trata con matices técnicos que debes conocer para no confundirte.

Qué significa NaN

NaN (Not a Number) es el estándar técnico en Pandas y NumPy para representar valores faltantes en columnas numéricas o de punto flotante. Es importante entender que, para la computadora, NaN es un objeto de tipo *float*.

- **Curiosidad:** En Python, si comparas `NaN == NaN`, el resultado es `False`. Por eso usamos funciones especiales para detectarlos.

Null y None

- **None:** Es el valor de "vacío" nativo de Python. Pandas suele convertir los `None` en `NaN` cuando carga datos en un `DataFrame` para mantener la eficiencia.
- **Null:** Es el término genérico que proviene del lenguaje de bases de datos (SQL). En la práctica, cuando hablamos de Pandas, usamos "nulo" y "faltante" (o NA) casi como sinónimos.

Cadenas vacías ("") vs. Faltantes

Este es un error común. Una cadena vacía (como un texto sin letras "") **no es un nulo** para Pandas. Es un texto válido que simplemente no tiene caracteres. Si un usuario dejó un espacio en blanco en un formulario, Pandas lo verá como un texto, no como un valor faltante, a menos que le indiquemos lo contrario durante la carga.

2. ¿Por qué aparecen valores faltantes?

Antes de abrir la caja de herramientas, debemos preguntarnos: **¿Por qué este dato no está aquí?** La respuesta cambiará totalmente tu estrategia.

1. **Captura incompleta:** El sensor falló por un segundo, o el encuestado prefirió no decir su salario.
2. **Errores de integración:** Al unir dos tablas (por ejemplo, Clientes y Compras), si un cliente nunca compró nada, sus columnas de "monto" aparecerán vacías. Esto no es un error, ¡es información valiosa! Nos dice quiénes son clientes inactivos.

3. **Datos no aplicables:** En una columna llamada "Nombre de la mascota", alguien que no tiene animales dejará el dato vacío por lógica, no por error.
4. **Diseño intencional:** A veces, los datos faltantes surgen porque el proceso de recolección cambió y antes no se pedía esa información.

Principio de oro: Nunca limpies sin entender el contexto. Si borras todos los nulos de una columna de "descuento aplicado", podrías estar borrando a todos los clientes que pagaron el precio completo.

3. Detección de valores faltantes

Para "ver" el problema en Pandas, tenemos tres herramientas fundamentales que ya hemos mencionado ligeramente, pero que ahora usaremos con precisión.

El vistazo general: `info()`

Como aprendiste anteriormente, `df.info()` te muestra cuántos valores "Non-Null" (no nulos) hay por columna. Si tu dataset tiene 1,000 filas y una columna solo tiene 800 "Non-Null", ya sabes que tienes 200 problemas que resolver ahí.

La lupa quirúrgica: `isna()` e `isnull()`

En Pandas, `isna()` e `isnull()` hacen exactamente lo mismo: devuelven una "máscara" de valores verdaderos (True) donde hay un nulo y falsos (False) donde no lo hay.

python

```
# Ejemplo conceptualimport pandas as pdimport numpy as npdata = {'Edad': [25, np.nan, 30], 'Ciudad': ['Madrid', 'Bogotá', None]}df = pd.DataFrame(data)print(df.isna())
```

Esto te devolverá una tabla de booleanos. Pero ver una tabla de True y False no es muy útil si tienes un millón de filas.

4. Cuantificación: ¿Qué tan grave es el problema?

Para tomar decisiones, necesitamos números. ¿Falta el 1% de los datos o el 60%?

Contar faltantes por columna

Podemos encadenar métodos. Recuerda que en Python, True vale 1 y False vale 0. Si sumamos los resultados de `isna()`, obtendremos el total de nulos por columna:

python

```
# Cuenta nulos por columnanulos_totales = df.isna().sum()
```

Cálculo de porcentajes

Si tienes 50 nulos en una columna, parece poco. Pero si el dataset tiene solo 60 filas, ¡es un desastre!

Siempre calcula el porcentaje:

python

```
# Porcentaje de faltantesporcentaje_nulos = (df.isna().sum() / len(df)) * 100
```

Regla empírica (pero no universal):

- **Menos del 5%:** Suele ser seguro eliminar las filas o imputar (rellenar) con métodos sencillos.
- **Entre 5% y 30%:** Requiere pensar una estrategia de imputación más sofisticada.
- **Más de 50%:** A veces es mejor descartar la columna entera, ya que tiene más "ruido" que información real.

5. Identificación de duplicados

Un duplicado no es solo una fila repetida; es una posible distorsión de la realidad. Si un sistema registró dos veces la misma venta de 2,000, lo cual es falso.

Detectar con `duplicated()`

Este método marca como `True` las filas que ya han aparecido antes.

- Por defecto, considera que una fila es duplicada si **todos** sus valores son idénticos a los de una fila anterior.
- Puedes usar el argumento `subset=['columna_id']` para buscar duplicados basándote solo en una columna específica (como el DNI o el ID de transacción).

Eliminar con `drop_duplicates()`

Este comando limpia el DataFrame. Puedes elegir quedarte con la primera aparición (`keep='first'`) o la última (`keep='last'`).

python

```
# Elimina duplicados basándose en el ID de usuario
df.drop_duplicates(subset='user_id', keep='first')
```

6. Criterios de limpieza: ¿Eliminar o Imputar?

Aquí es donde dejas de ser un programador y empiezas a ser un **Analista de Datos**. No hay una tecla mágica; hay criterios.

Estrategia A: Eliminar (`dropna`)

Eliminar filas (`df.dropna()`) es la opción más drástica. Es válida cuando:

1. Te sobran datos y perder un 2% no afecta estadísticamente.
2. El valor que falta es la "etiqueta" que quieres predecir (ej: quieres predecir si un cliente comprará, pero no sabes si el cliente de esa fila compró o no).

Estrategia B: Imputar (Rellenar)

Imputar significa sustituir el hueco por un valor estimado.

- **Media:** Útil para variables numéricas con una distribución "normal" (campana de Gauss) y sin valores extremos (outliers).
- **Mediana:** Más robusta si hay valores extremos. Si el salario de un grupo es de 10 personas y una es multimillonaria, la mediana representará mejor al grupo que el promedio.
- **Moda:** Ideal para variables categóricas (como "Color favorito" o "Estado Civil"). Rellenamos con el valor más frecuente.
- **Valor Constante:** A veces rellenamos con "Desconocido" o "0" para indicar que no tenemos el dato pero queremos conservar la fila.

El dilema de los duplicados

No todos los duplicados se borran.

- **Identidad:** Si son la misma persona comprando el mismo producto en el mismo segundo exacto, es un error del sistema: **BORRAR**.
- **Eventos:** Si es un cliente comprando el mismo artículo dos días seguidos, son dos eventos reales: **CONSERVAR**.
- **Consejo:** Antes de borrar, revisa si tienes una columna de "Timestamp" (marca de tiempo) o un ID único de transacción.

Síntesis: Tu flujo de trabajo de limpieza

Para esta unidad y para tu carrera profesional, sigue este mapa mental:

1. **Explora:** Usa `info()` y `isna().sum()` para dimensionar el problema.
2. **Contextualiza:** Pregúntate por qué falta el dato. ¿Es un error o una realidad del negocio?
3. **Mide:** Calcula porcentajes. No tomes decisiones basadas en números absolutos.
4. **Decide:**
 - ¿Puedo vivir sin esta fila? -> `dropna()`
 - ¿La variable es importante pero falta poco? -> Imputa (media/mediana/moda).
 - ¿Falta demasiada información? -> Evalúa borrar la columna.
5. **Deduplica:** Asegúrate de que cada evento sea único basándote en claves de negocio (IDs).

En la práctica de esta unidad, pondrás a prueba tu criterio analítico para decidir el destino de un dataset real. ¡Recuerda documentar siempre el porqué de tus decisiones!

Para Practicar (No entregable)

Propósito del ejercicio

Este ejercicio desarrolla tu capacidad de "diagnóstico clínico" de datos. Antes de transformar, debes entender qué tan saludable es tu conjunto de información y tomar decisiones ejecutivas sobre su limpieza.

Escenario

Utilizarás el dataset base que cargaste en el módulo anterior (puedes usar el dataset de ejemplo sugerido en tutorías o el de tu proyecto personal). Tu objetivo es presentar un reporte de calidad.

Pasos sugeridos

1. **Carga y Medición inicial:**
 - Utiliza Pandas para cargar tu archivo.
 - Aplica los métodos aprendidos para obtener el total de valores faltantes por columna.
 - Calcula qué porcentaje del total de filas representa cada ausencia.
2. **Detección de Duplicados:**
 - Busca filas exactamente iguales en todo el dataset.
 - Busca duplicados en columnas que deberían ser únicas (por ejemplo, IDs de usuario o números de factura).
3. **Análisis de Impacto:**
 - Para cada columna con nulos, pregúntate: ¿Qué significa que falte este dato? (Ej. Si falta 'edad', ¿puedo usar el promedio?).
 - Observa si los duplicados son errores de carga o registros válidos.
4. **Creación del Informe:**
 - Abre un documento (Word, Google Docs o Markdown) y estructúralo para que sea legible por un supervisor que no sabe programar.

Errores comunes a evitar

- **Confundir porcentajes:** Asegúrate de dividir el total de nulos entre el total de filas, no de columnas.
- **Imputar sin pensar:** No rellenes con "0" una columna de edad a menos que tengas una razón válida; un "0" distorsiona el promedio de edad mucho más que un nulo.
- **Entregar código:** Este checkpoint evalúa tu **criterio analítico**. Las tablas pueden ser capturas de pantalla de tu Jupyter Notebook, pero la parte central es la explicación de tu decisión.
✦ **Entregable de este módulo:** Pre-entrega — Limpieza y Análisis Exploratorio con Pandas (evaluable, suma al proyecto final). Su consigna y rúbrica están en la unidad de pre-entrega de este mismo módulo.

Transformaciones con map y apply para crear columnas útiles

Transformaciones con map y apply

En el flujo de trabajo de un Científico de Datos, rara vez los datos vienen listos para el análisis. A menudo necesitamos "traducir" etiquetas inconsistentes o calcular nuevos indicadores (métricas) que no existen originalmente en el dataset. En Pandas, las herramientas maestras para esto son `map` y `apply`.

1. ¿Por qué transformar datos?

Imagina que recibes una base de datos de ventas donde el país se registró como "ESP", "esp" y "España". Para un computador, estas son tres categorías distintas. Si intentas sacar el promedio de ventas por país, los resultados estarán fragmentados.

Transformar consiste en:

1. **Estandarizar:** Unificar criterios (ej. todos los países a "España").
2. **Enriquecer:** Crear "columnas derivadas" (ej. calcular el IVA a partir del precio).
3. **Categorizar:** Traducir valores numéricos a etiquetas de negocio (ej. "Gasto Alto", "Gasto Bajo").

2. El método map: La tabla de traducción

El método `.map()` se utiliza sobre una **Series** (una columna). Es ideal cuando tienes una correspondencia "uno a uno".

Ejemplo con diccionario:

python

```
# Queremos convertir códigos en nombres legibles
diccionario_genero = {'M': 'Masculino', 'F': 'Femenino'}
df['genero_completo'] = df['genero'].map(diccionario_genero)
```

- **Error común:** Si un valor en la columna original no está en el diccionario, `map` lo convertirá en NaN (nulo). ¡Asegúrate de cubrir todos los casos!

3. El método apply: Flexibilidad total

`apply` es la navaja suiza de Pandas. Permite ejecutar una función propia sobre cada elemento o sobre filas completas.

Apply en Series (Columna por columna)

Se usa para transformar una sola columna usando una lógica más compleja que un simple mapeo.

python

```
df['nombre_mayuscula'] = df['nombre'].apply(lambda x: x.upper())
```

Apply en DataFrames (Fila por fila)

Aquí es donde brilla para las **reglas de negocio**. Al usar `axis=1`, le decimos a Pandas: "pasa cada fila completa a la función".

python

```
def calcular_riesgo(fila):
    if fila['deuda'] > 1000 and fila['score_creditticio'] < 500:
        return 'Alto'
    return 'Bajo'
df['riesgo'] = df.apply(calcular_riesgo, axis=1)
```

4. Diferencias Clave

Método	Aplicación	Uso principal
map	Series	Sustitución simple (diccionarios).
apply	Series/DataFrame	Funciones complejas o lógica entre varias columnas.
applymap	DataFrame completo	Aplicar una función a todas las celdas de la tabla (ej. redondear todos los decimales).

5. Recomendaciones de Oro

1. **Prioriza Vectorización:** Si puedes hacer `df['A'] + df['B']`, no uses `apply`. Las operaciones directas de Pandas son mucho más rápidas.

2. **Valida tras transformar:** Usa siempre `df.head()` y `df['nueva_columna'].value_counts()` para verificar que la transformación hizo lo que esperabas.
3. **Cuidado con los Nulos:** Las funciones personalizadas a veces fallan si encuentran un NaN. Considera limpiar nulos antes de transformar.

Agrupar, resumir y comparar con GroupBy y tablas dinámicas

Agrupar, resumir y comparar con GroupBy y tablas dinámicas

Imagina que eres el gerente de una cadena de cafeterías con 50 sucursales en todo el país. Tienes una tabla con miles de filas que representan cada café vendido hoy. Tienes el nombre del producto, la sucursal, la hora y el precio. Si alguien te pregunta: "¿Cuál es la sucursal que más vendió?" o "¿Cuál es el promedio de ventas por tipo de bebida?", no puedes responder mirando fila por fila. Necesitas **agrupar** esa montaña de datos para extraer una conclusión que te permita tomar una decisión, como reforzar el personal en la sucursal más concurrida o crear una promoción para las bebidas menos vendidas.

En esta unidad, aprenderás a transformar datos detallados en resúmenes estratégicos utilizando dos de las herramientas más potentes de la librería Pandas: `groupby` y las tablas dinámicas (`pivot_table`). Estas herramientas son el "corazón" del análisis descriptivo en Ciencia de Datos.

1. El concepto de agrupación y resumen estadístico

Agrupar datos es el proceso de organizar registros individuales en categorías para calcular métricas que describan a cada grupo. Es pasar de lo **micro** (la venta de un café latte a las 10:15 AM en la Sucursal Norte) a lo **macro** (el total de ventas de la Sucursal Norte en todo el día).

Un **resumen estadístico**, por su parte, es un número único que representa a todo el grupo. Los más comunes son:

- **Suma:** ¿Cuánto dinero entró en total?
- **Promedio (Media):** ¿Cuál es el valor típico de una venta?
- **Conteo:** ¿Cuántas ventas se realizaron?
- **Máximo/Mínimo:** ¿Cuál fue la venta más cara o la más barata?

Sin la agrupación, los datos son solo ruido. Con la agrupación, los datos se convierten en **información de negocio**.

2. El flujo Split-Apply-Combine

Para entender cómo trabaja Pandas "bajo el capó" cuando agrupamos, debemos conocer el concepto desarrollado por Hadley Wickham llamado **Split-Apply-Combine** (Dividir-Aplicar-Combinar). Este es el modelo mental que debes tener cada vez que uses `groupby`.

1. **Split (Dividir):** Pandas toma el DataFrame original y lo separa en pedazos más pequeños basados en una variable (por ejemplo, "Sucursal"). Se crean tantas "cajas" como sucursales existan.
 2. **Apply (Aplicar):** Dentro de cada caja, aplicas una operación matemática. Por ejemplo, en la caja de la Sucursal Norte, sumas todos los precios; en la caja de la Sucursal Sur, haces lo mismo.
 3. **Combine (Combinar):** Finalmente, Pandas toma los resultados de cada caja (los totales de cada sucursal) y los une en una nueva tabla resumen mucho más pequeña que la original.
¿Por qué es importante entender esto? Porque a veces el paso "Apply" puede fallar si intentamos sumar cosas que no son números (como nombres de clientes). Como verás a continuación, debemos ser cuidadosos con las columnas que elegimos.
-

3. Selección de columnas y agregaciones comunes

Cuando agrupamos en Python, la sintaxis básica se ve

```
así: df.groupby('columna_agrupadora')['columna_a_resumir'].metodo()
```

Selección de columnas numéricas

Un error muy frecuente al iniciar es intentar promediar todo el DataFrame. Imagina que en tu tabla de ventas tienes una columna "Email del Cliente". ¿Cómo promedias un email? No se puede. Pandas intentará hacerlo y, en versiones recientes, te dará un error si no especificas que solo quieres operar con columnas numéricas.

Regla de oro: Siempre selecciona la columna numérica sobre la cual quieres hacer el cálculo antes de aplicar el método de resumen.

Métodos de agregación frecuentes

Pandas nos ofrece funciones integradas muy sencillas de usar:

- `.mean()`: Calcula el promedio. Ideal para tickets promedio de venta.
- `.sum()`: Suma todos los valores. Ideal para ingresos totales.
- `.count()`: Cuenta cuántos registros hay. Ideal para saber el volumen de pedidos.
- `.median()`: Calcula la mediana. Útil cuando tienes valores extremos (outliers) que distorsionan el promedio.
- `.min()` y `.max()`: Encuentran los valores extremos de cada grupo.

Ejemplo rápido: Si quieres saber el promedio de ventas por categoría de producto:

```
python
ventas_por_categoria = df.groupby('Categoria')['Precio'].mean()
```

4. Agregaciones múltiples con `agg()`

A veces, una sola métrica no es suficiente. Quizás necesitas saber el total de ventas y el promedio al mismo tiempo para comparar el volumen frente a la rentabilidad. Para esto usamos el método `.agg()` (de *aggregate*).

Este método nos permite pasar una lista de funciones:

```
python
resumen = df.groupby('Sucursal')['Venta'].agg(['sum', 'mean', 'count'])
```

El resultado será una tabla con tres columnas nuevas para cada sucursal. Esto es extremadamente potente porque permites que una sola línea de código responda tres preguntas de negocio distintas.

5. Agrupación por múltiples columnas

¿Qué pasa si queremos un nivel de detalle mayor? Por ejemplo: "Quiero el total de ventas por **Sucursal** y por **Tipo de Bebida** (Café, Té, Jugo)".

Podemos pasar una lista de columnas al `groupby`:

```
python
detalle = df.groupby(['Sucursal', 'Tipo_Bebida'])['Venta'].sum()
```

Esto crea una jerarquía. Verás la "Sucursal A", y debajo de ella, el desglose por tipo de bebida. Esto es vital para identificar, por ejemplo, si en una sucursal específica se venden mucho más jugos que cafés, lo cual podría indicar un perfil de cliente diferente (quizás una sucursal cerca de un gimnasio vs una en un centro financiero).

6. Pivot Tables: El poder de las tablas dinámicas

Si has usado Excel o Google Sheets, las **Tablas Dinámicas** te resultarán familiares. En Pandas, el método `pivot_table` hace exactamente lo mismo, pero de forma programática y reproducible.

¿Cuál es la diferencia con GroupBy? Mientras que `groupby` suele devolver un resultado vertical (fácil de procesar para la computadora), `pivot_table` genera una matriz bidimensional (fácil de leer para un humano).

Los componentes de una `pivot_table` son:

1. **Index:** La columna que irá en las filas (eje vertical).
2. **Columns:** La columna que irá en la parte superior (eje horizontal).
3. **Values:** La columna numérica que queremos resumir.
4. **Aggfunc:** La operación (promedio, suma, etc.). Por defecto es el promedio.

Ejemplo de escenario: Quieres ver las ventas totales cruzando **Región** (filas) y **Año** (columnas).

python

```
tabla = df.pivot_table(index='Region', columns='Anio', values='Venta',  
aggfunc='sum')
```

Esto te permite comparar visualmente y de forma inmediata cómo creció cada región año tras año en una sola vista.

7. Errores comunes y mejores prácticas

Como principiante, es normal encontrarse con algunos obstáculos. Aquí te mostramos cómo evitarlos:

1. **Confundir `count()` con `sum()`:** Es el error número uno. `count()` te dice cuántas filas hay (cuántas ventas hubo). `sum()` suma los valores de esas filas (cuánto dinero se recaudó). Si quieres contar clientes, usa `count()`. Si quieres sumar dinero, usa `sum()`.
2. **Olvidar los valores nulos:** Por defecto, Pandas ignora los valores faltantes (NaN) al calcular el promedio o la suma. Esto puede ser engañoso si el 90% de tus datos faltan. Siempre usa `df.info()` antes de agrupar para conocer el estado de tus datos.
3. **No "resetear" el índice:** Cuando agrupas, la columna por la que agrupaste se convierte en el "índice" de la tabla resultante. Si después quieres graficar o volver a filtrar, es recomendable usar `.reset_index()` para que vuelva a ser una columna normal.

python

```
resumen = df.groupby('Sucursal')['Venta'].sum().reset_index()
```

Síntesis y conexiones

Hemos aprendido que agrupar no es solo un comando técnico, sino una forma de pensar sobre los datos. El flujo **Split-Apply-Combine** nos permite desarmar un problema grande en partes pequeñas, resolverlo y volverlo a armar.

Ya sea que uses `groupby` para cálculos rápidos o `pivot_table` para reportes visuales, ahora tienes la capacidad de responder dudas complejas de negocio. En la siguiente unidad, aprenderás cómo estas tablas resumidas pueden conectarse con otras fuentes de información mediante procesos de unión (*merging*), lo que te permitirá enriquecer aún más tus análisis.

Recuerda: **Un buen científico de datos no solo calcula promedios; entiende qué historia cuentan esos promedios sobre el mundo real.** (Analogía: La nota media de una clase no define a cada alumno, pero te dice muy rápido si el profesor está logrando que el grupo entienda la materia).

Para Practicar (No entregable)

Propósito del ejercicio

Este ejercicio te permitirá aplicar las técnicas de agrupación y resumen en un contexto de negocio real, utilizando el dataset que has venido limpiando y transformando en las unidades anteriores. El objetivo es pasar de los datos "sucios" a información que un gerente podría usar para tomar decisiones.

Instrucciones paso a paso

1. **Preparación:** Utiliza el DataFrame que limpiaste en la unidad 1 (manejo de nulos) y enriqueciste en la unidad 2 (columnas derivadas). Asegúrate de tener al menos una columna categórica (ej: 'Categoria', 'Sucursal', 'Region') y una numérica (ej: 'Ventas', 'Ingresos').
2. **Agrupación de Negocio (GroupBy):**
 - Selecciona una dimensión principal de tu negocio (por ejemplo, la categoría de producto o la región).
 - Utiliza el método `.groupby()` junto con `.agg()` para calcular en una sola tabla: el total de ventas (`sum`), el promedio de venta por transacción (`mean`) y el número de transacciones totales (`count`).
 - Dale nombres claros a las columnas resultantes (ej: 'Total_Recaudado', 'Promedio_Venta', 'Cantidad_Ventas').
3. **Construcción de Tabla Dinámica (Pivot Table):**
 - Crea una tabla dinámica que cruce dos dimensiones (por ejemplo, 'Region' en las filas y 'Categoria' en las columnas).
 - Resume el valor de las ventas en el centro de la tabla usando la suma como función de agregación.
4. **Análisis y Conclusión:**
 - Observa los resultados. ¿Cuál es el grupo que genera más ingresos? ¿El grupo con más ventas es el mismo que tiene el promedio más alto?
 - Escribe una conclusión de 2 a 3 líneas que sea "accionable". Ejemplo: *"La Región Norte tiene el promedio de venta más alto, pero la menor cantidad de transacciones; se recomienda una campaña de marketing para atraer más clientes en esa zona."*
5. **Exportación:**
 - Exporta los resultados a un archivo Excel o Google Sheets. Organiza el archivo con pestañas claras para que una persona que no sabe programación pueda entenderlo.

Errores comunes a evitar

- No confundas el volumen (`count`) con el valor (`sum`).
 - Asegúrate de que la tabla dinámica no tenga demasiados "campos vacíos" que dificulten la lectura; si los tiene, intenta agrupar por categorías con más datos.
-

Fechas, series temporales y resampling para análisis por periodo

En el mundo de los datos, el tiempo no es solo un número o un texto; es una **dimensión fundamental**. Una serie temporal es, sencillamente, una colección de datos registrados en momentos sucesivos. Pensa en las ventas diarias de un negocio, la temperatura por hora o el precio de una acción en la bolsa. Sin embargo, trabajar con fechas suele ser caótico: el texto puede venir en formatos distintos ("01/12/2023" vs "2023-12-01"), hay meses con distintas cantidades de días y años bisiestos. Pandas nos ofrece herramientas poderosas para que el tiempo trabaje a nuestro favor.

1. Conversión de texto a Datetime

Cuando cargamos un archivo CSV, las fechas suelen leerse como simple texto (`string` u `object` en Pandas). Para que Pandas pueda "entender" la lógica del calendario, debemos convertirlas

usando `pd.to_datetime()`.

python

```
# Convierte una columna de texto a objetos de fecha realesdf['fecha'] = pd.to_datetime(df['fecha'])
```

2. El Índice Temporal: Tu mejor aliado

Para realizar análisis avanzados de tiempo, la mejor práctica es convertir nuestra columna de fechas en el **índice** del DataFrame. Esto nos permite:

- Acceder a datos por periodos (ej: `df['2023-01']` para ver todo enero).
- Realizar operaciones de **Resampling** (cambio de frecuencia).

3. Resampling: Cambiando el "zoom" de tus datos

¿Qué pasa si tienes datos de ventas minuto a minuto pero tu jefe quiere un reporte mensual? Aquí entra `resample()`. Es como un `groupby` especializado en tiempo.

Existen dos tipos principales:

- **Downsampling:** Reducir la frecuencia (ej: de diario a mensual). Calculamos una estadística (suma, promedio) para resumir el periodo.
- **Upsampling:** Aumentar la frecuencia (ej: de anual a mensual). Requiere decidir cómo "rellenar" los huecos creados.

Ejemplo de códigos de frecuencia comunes:

- D: Diario
- W: Semanal
- ME: Fin de mes (Month End)
- YE: Fin de año (Year End)

Errores comunes a evitar

1. **No ordenar los datos:** Antes de remuestrear, siempre asegúrate de que tus fechas estén en orden cronológico con `df.sort_index()`. Un índice desordenado puede dar resultados impredecibles.
2. **Operar con texto:** Si intentas sumar o promediar fechas que no convertiste a `datetime`, Python arrojará un error de tipos.
3. **Confundir sumas con promedios:** Al pasar de diario a mensual, ¿quieres saber la venta *total* (`.sum()`) o la venta *promedio* por día en ese mes (`.mean()`)? Elige tu función de agregación con cuidado según la pregunta de negocio.

Manipulación de Datos: Pandas

El Arte de la Manipulación de Datos con Pandas: Tu Superpoder Analítico

Imagina que trabajas en el departamento de análisis de una gran plataforma de streaming como Netflix. Cada mañana, recibes un archivo con millones de filas que contienen clics, tiempos de visualización, valoraciones de usuarios y géneros de películas. Tienes una misión: descubrir qué género fue el más popular ayer para recomendarlo en la pantalla de inicio.

Si intentarás abrir este archivo en una hoja de cálculo tradicional, probablemente tu computadora se congelaría. Si intentarás procesarlo fila por fila usando Python básico, tardarías horas en escribir el código

y aún más en ejecutarlo. Aquí es donde entra **Pandas**, la herramienta que transforma el caos de los datos "sucios" en información estructurada, limpia y lista para la toma de decisiones.

En esta unidad final, consolidaremos todo lo aprendido en el módulo, entendiendo a Pandas no solo como una librería, sino como un flujo de trabajo profesional que te permite interrogar a los datos y obtener respuestas claras.

1. ¿Por qué existe Pandas? De la hoja de cálculo a la programación

Pandas fue creado por **Wes McKinney** en el sector financiero hacia el año 2008. En ese entonces, los analistas necesitaban algo que tuviera la flexibilidad de Python pero la estructura de una base de datos o una hoja de cálculo. El nombre proviene de *Panel Data* (datos de panel), un término de econometría para conjuntos de datos que observan los mismos elementos a lo largo del tiempo.

La diferencia fundamental: Escala y Repetibilidad

La gran debilidad de herramientas visuales como Excel es que, si mañana recibes un archivo nuevo, tienes que volver a hacer clic en los mismos botones, aplicar los mismos filtros y arrastrar las mismas fórmulas. En Pandas, escribes el "recetario" una vez. No importa si el archivo tiene 100 filas o 1,000,000; el código se ejecuta exactamente igual, garantizando que no haya errores humanos en el proceso.

Concepto Clave: Pandas no piensa en "celdas", piensa en **estructuras etiquetadas**. Esto significa que siempre sabemos qué significa cada dato gracias a sus encabezados (columnas) y sus identificadores (índice).

2. Los Pilares de Pandas: DataFrames y Series

Para dominar Pandas, debes tener una imagen mental clara de sus dos estructuras principales.

El DataFrame: Tu tabla de comando

El `DataFrame` es una estructura bidimensional. Piensa en él como una tabla de base de datos o una hoja de Excel. Tiene dos ejes:

1. **Eje 0 (Filas):** Representan registros u observaciones (ej. una venta, un usuario, un sensor).
2. **Eje 1 (Columnas):** Representan variables o atributos (ej. precio, nombre, fecha).

La Series: El ADN de los datos

Una `Series` es una sola columna. Es unidimensional y, lo más importante, tiene un **Índice**.

- **Error común:** Muchos estudiantes piensan que una Serie es solo una lista de Python. La diferencia es que una Serie tiene etiquetas. Si tienes una lista de precios, no sabes a qué producto pertenece cada uno. En una Serie de Pandas, el precio puede estar etiquetado con el ID "Prod_01".
-

3. El Flujo de Trabajo Profesional (Pipeline)

Un científico de datos no lanza comandos al azar. Sigue un proceso lógico que Pandas facilita enormemente. Este es el camino que recorren los datos desde que llegan "crudos" hasta que se convierten en conocimiento:

fase 1: Ingesta e Inspección

Primero, traemos los datos de fuentes como CSV, Excel, SQL o JSON.

- **`.head()` y `.tail()`:** Nos dan una "probada" de los datos.

- **.info()**: Es vital. Nos dice cuántos valores nulos hay y qué tipo de datos tenemos (¿son números o texto?). Si intentas sumar una columna que Pandas detectó como texto, tu análisis fallará.

Fase 2: Limpieza (Tratamiento de NaN)

En el mundo real, los datos son imperfectos. Faltan nombres, hay sensores que fallaron o usuarios que no completaron su perfil. Estos huecos se representan como NaN (Not a Number).

- **Decidir qué limpiar:** Como vimos anteriormente, puedes borrar filas con nulos (`dropna`) si son pocas, o rellenarlas (`fillna`) con un promedio si no quieres perder información valiosa.

Fase 3: Transformación y Vectorización

Aquí es donde Pandas brilla sobre Python básico. En lugar de hacer un bucle `for` para aplicar un descuento del 10% a cada fila de una columna de precios, hacemos: `df['precio'] * 0.9`. Esto se llama **operación vectorizada**. Es miles de veces más rápida porque Pandas (apoyándose en la librería NumPy) realiza el cálculo en todos los elementos simultáneamente a nivel de memoria.

4. Agregación y el Modelo "Split-Apply-Combine"

Uno de los conceptos más potentes es el `groupby`. Para entenderlo, usemos el modelo mental de **"Separar, Aplicar y Combinar"**:

1. **Separar (Split):** Divides tus datos en grupos basados en una columna (ej. agrupar ventas por "País").
2. **Aplicar (Apply):** Calculas algo para cada grupo independientemente (ej. sumar las ventas de cada país).
3. **Combinar (Combine):** Pandas vuelve a juntar esos resultados en una nueva tabla resumen.

Este proceso es el que permite a empresas como **J.P. Morgan** o **Goldman Sachs** analizar miles de transacciones por segundo para obtener promedios diarios o máximos históricos por activo financiero.

5. Combinación de Fuentes: Uniendo Rompecabezas

Rara vez toda la información está en un solo sitio. Podrías tener un archivo con las "Ventas" y otro con la "Información de Clientes".

- **Merge:** Es como el `JOIN` de SQL. Buscas una columna común (ej. `ID_Cliente`) y fusionas las tablas lateralmente.
- **Concat:** Es como apilar hojas de papel. Pones una tabla debajo de la otra (ej. ventas de Enero debajo de ventas de Febrero).

Punto de atención: Un error frecuente es confundir ambos. Recuerda: `Merge` añade **columnas** nuevas sobre los mismos registros; `Concat` añade **filas** nuevas del mismo tipo.

6. Errores Comunes y Mejores Prácticas

Incluso los expertos cometen estos errores cuando empiezan. Identificarlos te ahorrará horas de frustración:

1. **La trampa del Índice:** El índice de Pandas no es una columna normal; es el sistema de direcciones. Si filtras datos y reinicias el índice, asegúrate de saber si quieres conservar el antiguo como una columna o descartarlo.
2. **Copias vs. Vistas:** A veces, al filtrar un `DataFrame`, Pandas no crea una tabla nueva, sino que te muestra una "ventana" a la original. Si intentas cambiar algo en esa ventana, Pandas te dará un aviso (`SettingWithCopyWarning`). La mejor práctica es usar `.copy()` si realmente quieres una tabla independiente.
3. **No tratar los nulos al inicio:** Si intentas hacer cálculos estadísticos sin haber gestionado los NaN, tus resultados pueden ser sesgados o causar errores en funciones matemáticas.
4. **Alineación Automática:** Pandas es muy inteligente. Si sumas dos Series, las sumará **por su etiqueta**, no por su posición física. Si la Serie A tiene el índice [1, 2] y la Serie B tiene [2, 1], Pandas buscará el "1" en

ambas y los sumará, aunque estén en orden diferente. Esto es una ventaja, pero confunde a quienes vienen de usar listas simples.

7. Aplicaciones en la Industria Real

¿Dónde se usa esto realmente?

- **Comercio Electrónico:** Para detectar valores atípicos (outliers) en precios que podrían indicar errores en la carga de productos.
- **Logística:** Para calcular tiempos de entrega promedio uniendo tablas de rutas, tráfico y conductores.
- **Salud:** Para limpiar registros de pacientes, unificando formatos de fechas y manejando datos clínicos faltantes en ensayos de medicamentos.

Síntesis de la Unidad

Pandas es el puente entre los datos crudos y la ciencia de datos aplicada. A través de este módulo, has aprendido a:

1. **Cargar y explorar** estructuras como DataFrames y Series.
 2. **Sanear** datos mediante la gestión de nulos y duplicados.
 3. **Transformar** columnas eficientemente sin usar bucles lentos.
 4. **Resumir** grandes volúmenes de información con agregaciones.
 5. **Conectar** diferentes fuentes para tener una visión 360 de un problema.
- Dominar estas herramientas no solo te hace más rápido; te permite realizar análisis que serían imposibles de otro modo, dándote la capacidad de extraer valor de cualquier conjunto de datos que encuentres en tu carrera profesional.

Práctica sugerida: Consolidación del Pipeline

Para poner en práctica todo lo aprendido de forma integrada, realiza el siguiente ejercicio diseñado para simular un escenario real de limpieza y transformación.

Fase	Tarea	Propósito
Exploración	Carga un set de datos y usa <code>.info()</code>	Identificar qué columnas necesitan limpieza inmediata.
Limpieza	Trata nulos en una columna numérica usando la mediana	Evitar que los valores faltantes arruinen tus promedios.
Ingeniería	Crea una nueva columna basada en una condición (ej. <code>precio > 100</code>)	Categorizar datos para un análisis más humano.
Análisis	Agrupar por una categoría y calcula la suma y el conteo	Obtener respuestas de negocio claras (ej. ¿cuántas ventas hubo por región?).

¡Felicidades por completar este módulo! Ahora tienes los cimientos necesarios para enfrentarte a desafíos reales de manipulación de datos con la confianza de un profesional.

Para practicar (no entregable):

Ejercicio de Consolidación: Diseñando un Pipeline de Datos

Este ejercicio no requiere programación, sino capacidad analítica para estructurar la solución a un problema de datos real.

Escenario: Trabajas para una cadena de tiendas de electrónica. Tienes dos archivos:

1. `Ventas.csv`: Contiene `ID_Producto`, `Fecha`, `Cantidad`, y `Precio_Unitario`. Algunas filas tienen la `Cantidad` vacía.
2. `Productos.csv`: Contiene `ID_Producto`, `Nombre_Producto` y `Categoría`.

Tu tarea: Describe paso a paso cómo usarías Pandas para obtener un informe que muestre "**El ingreso total por Categoría de producto**".

Pasos a seguir:

1. **Diagrama de Flujo:** Crea un esquema (puedes usar herramientas como Lucidchart, Canva o incluso dibujo a mano) que muestre el orden de las operaciones: Carga -> Limpieza -> Unión -> Transformación -> Agregación.
2. **Justificación Técnica:**
 - ¿Qué técnica usarías para los valores nulos en la columna 'Cantidad' de las ventas y por qué?
 - ¿Qué tipo de unión (merge/concat) utilizarías para juntar ambos archivos y cuál sería la "llave" o columna común?
 - Describe el proceso de `groupby` que realizarías para obtener el ingreso por categoría.
3. **Control de Errores:** Menciona un posible error que podría ocurrir en este flujo (ej. tipos de datos incorrectos) y cómo lo verificarías con los comandos de inspección de Pandas.

Pre-Entrega: Limpieza y Análisis Exploratorio de un Dataset Real

Pre-Entrega: Limpieza y Análisis Exploratorio de un Dataset Real

El objetivo de esta práctica es consolidar tus habilidades de manipulación de datos con **Pandas** mediante la creación de un script de limpieza y análisis inicial. Deberás trabajar sobre un dataset de tu elección (o uno sugerido de fuentes como Kaggle o datos gubernamentales) que presente desafíos técnicos como valores nulos, duplicados o tipos de datos incorrectos.

